# Machine Learning: A Comprehensive Textbook

Solutions to Chapter 15 Exercises

Generative Models: VAEs, GANs, Diffusion, and Beyond

Solutions Manual

**Exercise 15.1 — ELBO Derivation**

Derive the VAE ELBO from $\log p_\theta(x)$ using Jensen's inequality. Show that $\log p_\theta(x) = L_{\text{ELBO}} + D_{\text{KL}}(q_\phi(z \mid x) \| p_\theta(z \mid x))$.

**Solution.** **Deriving the ELBO via Jensen's inequality.** We want a tractable lower bound on the (generally intractable) marginal log-likelihood $\log p_\theta(x) = \log \int p_\theta(x, z)\, dz$. Introduce an arbitrary variational distribution $q_\phi(z \mid x)$ (the approximate posterior/encoder) and rewrite by multiplying and dividing:

$$\log p_\theta(x) = \log \int p_\theta(x, z)\, dz = \log \int q_\phi(z \mid x)\, \frac{p_\theta(x, z)}{q_\phi(z \mid x)}\, dz = \log \mathbb{E}_{z \sim q_\phi(z|x)} \left[ \frac{p_\theta(x, z)}{q_\phi(z \mid x)} \right].$$

Since log is a concave function, Jensen's inequality (in the form $\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$ for concave log, the reverse-direction analogue of the convex-function Jensen's inequality proven in Exercise 2.9) gives

$$\log p_\theta(x) = \log \mathbb{E}_{q_\phi} \left[ \frac{p_\theta(x, z)}{q_\phi(z \mid x)} \right] \geq \mathbb{E}_{q_\phi} \left[ \log \frac{p_\theta(x, z)}{q_\phi(z \mid x)} \right] =: L_{\text{ELBO}}(\theta, \phi; x). \qquad \blacksquare$$

This defines the **Evidence Lower BOund (ELBO)**, $L_{\text{ELBO}} = \mathbb{E}_{q_\phi}[\log p_\theta(x, z) - \log q_\phi(z \mid x)]$, a tractable quantity (computable via sampling from $q_\phi$ and evaluating the known densities $p_\theta, q_\phi$) that lower-bounds the intractable true log-likelihood.

**Exact decomposition (showing the ELBO gap is exactly the KL term).** Start from the ELBO expression and use $p_\theta(x, z) = p_\theta(x)\, p_\theta(z \mid x)$ (chain rule of probability):

$$L_{\text{ELBO}} = \mathbb{E}_{q_\phi} \left[ \log \frac{p_\theta(x)\, p_\theta(z \mid x)}{q_\phi(z \mid x)} \right] = \mathbb{E}_{q_\phi}[\log p_\theta(x)] + \mathbb{E}_{q_\phi} \left[ \log \frac{p_\theta(z \mid x)}{q_\phi(z \mid x)} \right].$$

The first term is $\log p_\theta(x)$ (a constant with respect to the expectation over $z$, since it doesn't depend on $z$), and the second term is exactly $-D_{\text{KL}}(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x))$ (by definition of KL divergence, Exercise 2.8/2.9, note the sign: $\mathbb{E}_q[\log(p/q)] = -\mathbb{E}_q[\log(q/p)] = -D_{\text{KL}}(q\|p)$). So:

$$L_{\text{ELBO}} = \log p_\theta(x) - D_{\text{KL}}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big),$$

which rearranges directly to

$$\log p_\theta(x) = L_{\text{ELBO}} + D_{\text{KL}}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big). \qquad \blacksquare$$

Since $D_{\text{KL}}(\cdot\|\cdot) \geq 0$ always (Exercise 2.9), this decomposition immediately re-derives the ELBO inequality $L_{\text{ELBO}} \leq \log p_\theta(x)$ as a corollary (consistent with the Jensen's-inequality derivation above), and additionally shows *exactly* what the gap between the bound and the true log-likelihood is: the KL divergence between the approximate posterior $q_\phi(z \mid x)$ and the true (intractable) posterior $p_\theta(z \mid x)$. This has an important practical consequence: maximizing the ELBO with

respect to $\phi$ (for fixed $\theta$) is *exactly equivalent* to minimizing $D_{\mathrm{KL}}(q_\phi(z \mid x) \| p_\theta(z \mid x))$ (since $\log p_\theta(x)$ doesn't depend on $\phi$), meaning the ELBO objective simultaneously (a) tightens the bound on the log-likelihood by making $q_\phi$ a better approximation to the true posterior, and (b) (via the $\theta$-gradient) directly increases a lower bound on the actual data log-likelihood — precisely why maximizing the ELBO is a sound and widely-used training objective for latent-variable generative models like VAEs.

### Exercise 15.2 — KL for Gaussian VAE

Derive the closed-form KL divergence $D_{\mathrm{KL}}(\mathcal{N}(\mu, \mathrm{diag}(\sigma^2)) \| \mathcal{N}(0, I)) = -\frac{1}{2}\sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$.

**Solution.** Because both distributions are diagonal-covariance Gaussians, the KL divergence factorizes as a sum over independent coordinates $j = 1, \ldots, d$ (a standard property: KL between product distributions with matching factorization equals the sum of per-coordinate KLs). So it suffices to compute the 1D case, $D_{\mathrm{KL}}(\mathcal{N}(\mu_j, \sigma_j^2) \,\|\, \mathcal{N}(0, 1))$, and sum over $j$.

Using the general Gaussian-KL formula derived in Exercise 2.8(a), $D_{\mathrm{KL}}(\mathcal{N}(\mu_1, \sigma_1^2) \| \mathcal{N}(\mu_2, \sigma_2^2)) = \ln \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$, with $\mu_1 = \mu_j, \sigma_1 = \sigma_j$ (the approximate posterior) and $\mu_2 = 0, \sigma_2 = 1$ (the standard normal prior):

$$D_{\mathrm{KL}}(\mathcal{N}(\mu_j, \sigma_j^2) \,\|\, \mathcal{N}(0, 1)) = \ln \frac{1}{\sigma_j} + \frac{\sigma_j^2 + \mu_j^2}{2} - \frac{1}{2} = -\ln \sigma_j + \frac{\sigma_j^2 + \mu_j^2}{2} - \frac{1}{2}.$$

Multiply the $-\ln \sigma_j$ term by $2/2$ and use $-2 \ln \sigma_j = -\ln \sigma_j^2$:

$$= -\frac{1}{2} \ln \sigma_j^2 + \frac{\sigma_j^2}{2} + \frac{\mu_j^2}{2} - \frac{1}{2} = -\frac{1}{2}\Big( \ln \sigma_j^2 - \sigma_j^2 - \mu_j^2 + 1 \Big) = -\frac{1}{2}\Big( 1 + \ln \sigma_j^2 - \mu_j^2 - \sigma_j^2 \Big).$$

Summing over all $d$ coordinates (using the diagonal-covariance factorization property noted above):

$$D_{\mathrm{KL}}\big(\mathcal{N}(\mu, \mathrm{diag}(\sigma^2)) \,\|\, \mathcal{N}(0, I)\big) = \sum_{j=1}^{d} D_{\mathrm{KL}}(\mathcal{N}(\mu_j, \sigma_j^2) \| \mathcal{N}(0, 1)) = -\frac{1}{2} \sum_{j=1}^{d} \Big( 1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2 \Big). \qquad \blacksquare$$

This closed-form expression is exactly the regularization term used in the VAE's ELBO loss (Exercise 15.1): since the prior $p(z) = \mathcal{N}(0, I)$ is standard normal and the approximate posterior $q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \mathrm{diag}(\sigma_\phi^2(x)))$ is diagonal Gaussian (the standard VAE parameterization, with the encoder network outputting $\mu_\phi(x)$ and $\log \sigma_\phi^2(x)$), this closed-form KL term (rather than requiring Monte Carlo estimation) can be computed exactly and differentiably, which is precisely why the standard (diagonal-Gaussian-prior-and-posterior) VAE is computationally efficient to train — no sampling is needed for the KL term of the loss, only for the reconstruction term (via the reparameterization trick).

### Exercise 15.3 — GAN Optimal Discriminator

Prove that $D^*(x) = p_{\mathrm{data}}(x)/(p_{\mathrm{data}}(x) + p_g(x))$ and show that $V(G, D^*) = -\log 4 + 2 D_{JS}(p_{\mathrm{data}} \| p_g)$.

**Solution.** **Optimal discriminator.** The GAN minimax value function is

$$V(G, D) = \mathbb{E}_{x \sim p_{\mathrm{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))] = \int p_{\mathrm{data}}(x) \log D(x)\, dx + \int p_g(x) \log(1 - D(x))\, dx,$$

where the second equality uses a change of variables from $z$ to $x = G(z)$, with $p_g$ the induced distribution of generated samples. Combine into a single integral:

$$V(G, D) = \int \Big[ p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x)) \Big] dx.$$

For *fixed* $G$ (hence fixed $p_g$), we maximize the integrand pointwise, for each $x$, as a function of $D(x) =: y \in [0, 1]$: $f(y) = a \log y + b \log(1-y)$ where $a = p_{\text{data}}(x) \geq 0, b = p_g(x) \geq 0$. Differentiate:

$$f'(y) = \frac{a}{y} - \frac{b}{1-y} = 0 \implies a(1-y) = by \implies a = y(a+b) \implies y^* = \frac{a}{a+b}.$$

(Second derivative $f''(y) = -a/y^2 - b/(1-y)^2 < 0$ confirms a maximum, assuming $a, b$ not both zero.) So the pointwise-optimal discriminator is

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}. \qquad \blacksquare$$

This has the natural interpretation of the Bayes-optimal classifier distinguishing real ($p_{\text{data}}$) from fake ($p_g$) samples under a uniform prior over the two classes.

**Value at the optimal discriminator.** Substitute $D^*(x)$ back into $V(G, D)$:

$$V(G, D^*) = \mathbb{E}_{x \sim p_{\text{data}}} \left[ \log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \right] + \mathbb{E}_{x \sim p_g} \left[ \log \frac{p_g(x)}{p_{\text{data}}(x) + p_g(x)} \right].$$

Multiply and divide each fraction's denominator by 2: $\frac{p_{\text{data}}(x)}{p_{\text{data}}(x)+p_g(x)} = \frac{1}{2} \cdot \frac{p_{\text{data}}(x)}{(p_{\text{data}}(x)+p_g(x))/2}$, so $\log \frac{p_{\text{data}}}{p_{\text{data}}+p_g} = -\log 2 + \log \frac{p_{\text{data}}}{(p_{\text{data}}+p_g)/2}$. Applying this to both terms:

$$V(G, D^*) = -\log 2 + \mathbb{E}_{p_{\text{data}}} \left[ \log \frac{p_{\text{data}}(x)}{m(x)} \right] - \log 2 + \mathbb{E}_{p_g} \left[ \log \frac{p_g(x)}{m(x)} \right],$$

where $m(x) := \dfrac{p_{\text{data}}(x) + p_g(x)}{2}$ is the mixture distribution. Recognize the two expectation terms as exactly KL divergences:

$$\mathbb{E}_{p_{\text{data}}} \left[ \log \frac{p_{\text{data}}}{m} \right] = D_{\text{KL}}(p_{\text{data}} \| m), \qquad \mathbb{E}_{p_g} \left[ \log \frac{p_g}{m} \right] = D_{\text{KL}}(p_g \| m).$$

So

$$V(G, D^*) = -2 \log 2 + D_{\text{KL}}(p_{\text{data}} \| m) + D_{\text{KL}}(p_g \| m).$$

By definition, the **Jensen-Shannon divergence** is exactly $D_{JS}(p_{\text{data}} \| p_g) := \frac{1}{2} D_{\text{KL}}(p_{\text{data}} \| m) + \frac{1}{2} D_{\text{KL}}(p_g \| m)$ (the average KL of each distribution to their mixture), i.e. $D_{\text{KL}}(p_{\text{data}} \| m) + D_{\text{KL}}(p_g \| m) = 2 D_{JS}(p_{\text{data}} \| p_g)$. Substituting, and using $-2 \log 2 = -\log 4$:

$$V(G, D^*) = -\log 4 + 2 D_{JS}(p_{\text{data}} \| p_g). \qquad \blacksquare$$

**Significance.** Since $D_{JS} \geq 0$ always (it is a bounded, symmetrized version of KL, itself non-negative by Exercise 2.9), the minimum possible value of $V(G, D^*)$ over choices of $G$ is $-\log 4$, achieved exactly when $D_{JS}(p_{\text{data}} \| p_g) = 0$, i.e. when $p_g = p_{\text{data}}$ (the generator perfectly matches the true data distribution, at which point the optimal discriminator is $D^*(x) = 1/2$ everywhere — unable to distinguish real from fake at all better than chance). This confirms that the GAN minimax game, at its saddle point (optimal $D$, and $G$ minimizing this resulting value), is precisely minimizing the Jensen-Shannon divergence between the generator's distribution and the true data distribution — providing the original theoretical justification (Goodfellow et al. 2014) for why adversarial training should, in principle, drive $p_g \to p_{\text{data}}$.

### Exercise 15.4 — DDPM Forward Process

Prove by induction that $q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$ from the Markov chain definition.

**Solution.** **Setup.** The DDPM forward (noising) process is defined as a Markov chain with single-step transitions

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\, x_{t-1},\ (1-\alpha_t)I\big),$$

where $\alpha_t \in (0,1)$ is the (per-step) signal-retention coefficient, and $\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$ is the cumulative product. We want to show $q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$ for all $t \geq 1$, by induction on $t$.

**Base case, $t = 1$.** $\bar{\alpha}_1 = \alpha_1$ (product of a single term), and directly by definition, $q(x_1 \mid x_0) = \mathcal{N}(\sqrt{\alpha_1}x_0, (1-\alpha_1)I) = \mathcal{N}(\sqrt{\bar{\alpha}_1}x_0, (1-\bar{\alpha}_1)I)$ — matches the claimed formula exactly.

**Inductive step.** Assume the claim holds at step $t-1$: $q(x_{t-1} \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}}x_0, (1-\bar{\alpha}_{t-1})I)$, i.e. we can write $x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{1-\bar{\alpha}_{t-1}}\, \epsilon_{t-1}$ for $\epsilon_{t-1} \sim \mathcal{N}(0, I)$ (the standard Gaussian reparameterization). We want to establish the claim at step $t$.

By the Markov property and the single-step transition, $x_t \mid x_{t-1} \sim \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, (1-\alpha_t)I)$, i.e. we can write $x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon'_t$ for a fresh, independent $\epsilon'_t \sim \mathcal{N}(0, I)$ (independent of $\epsilon_{t-1}$, since it represents new noise injected at this specific step). Substitute the inductive hypothesis's expression for $x_{t-1}$:

$$x_t = \sqrt{\alpha_t}\Big(\sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1-\bar{\alpha}_{t-1}}\, \epsilon_{t-1}\Big) + \sqrt{1-\alpha_t}\, \epsilon'_t = \sqrt{\alpha_t\bar{\alpha}_{t-1}}\, x_0 + \sqrt{\alpha_t(1-\bar{\alpha}_{t-1})}\, \epsilon_{t-1} + \sqrt{1-\alpha_t}\, \epsilon'_t.$$

The coefficient of $x_0$ is $\sqrt{\alpha_t\bar{\alpha}_{t-1}} = \sqrt{\bar{\alpha}_t}$ (using $\bar{\alpha}_t = \alpha_t\bar{\alpha}_{t-1}$ by definition of the cumulative product). The remaining two terms are a sum of two *independent* zero-mean Gaussian vectors, $\sqrt{\alpha_t(1-\bar{\alpha}_{t-1})}\, \epsilon_{t-1} + \sqrt{1-\alpha_t}\, \epsilon'_t$; the sum of independent Gaussians is itself Gaussian, with variance equal to the sum of the individual variances:

$$\mathrm{Var}\Big[\sqrt{\alpha_t(1-\bar{\alpha}_{t-1})}\epsilon_{t-1} + \sqrt{1-\alpha_t}\epsilon'_t\Big] = \alpha_t(1-\bar{\alpha}_{t-1})\, I + (1-\alpha_t)\, I = \big[\alpha_t - \alpha_t\bar{\alpha}_{t-1} + 1 - \alpha_t\big] I = \big[1 - \alpha_t\bar{\alpha}_{t-1}\big] I = (1-\bar{\alpha}_t)I,$$

again using $\alpha_t\bar{\alpha}_{t-1} = \bar{\alpha}_t$. So the combined noise term is distributed as $\mathcal{N}(0, (1-\bar{\alpha}_t)I)$, and hence

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \tilde{\epsilon}, \qquad \tilde{\epsilon} \sim \mathcal{N}(0, (1-\bar{\alpha}_t)I),$$

i.e.

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)I\big),$$

exactly matching the claimed formula at step $t$. This completes the inductive step, and by induction the formula $q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$ holds for all $t \geq 1$. ■

**Significance.** This closed-form result is what makes DDPM training tractable and efficient: rather than needing to simulate the full $t$-step Markov chain sequentially to obtain a noisy sample $x_t$ from a clean image $x_0$, one can sample $x_t$ *directly in a single step* via $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$ for a single $\epsilon \sim \mathcal{N}(0, I)$ — this is precisely what allows efficient, parallelizable training of the denoising network across randomly-sampled time steps $t$ (rather than requiring an expensive sequential forward pass through the chain for every training example).

### Exercise 15.5 — MoE Load Balancing

Show that without the auxiliary loss, a router with softmax can collapse to always routing to the same expert. Why does the gradient of the loss w.r.t. the router vanish in this case?

**Solution.** **The collapse mechanism.** In a Mixture-of-Experts (MoE) layer, a router computes $p(e \mid x) = \text{softmax}(W_r x)_e$ over $E$ experts, and (in a top-1 or top-$k$ routing scheme) sends each input $x$ to its highest-probability expert(s). Suppose that, purely due to random initialization or early-training dynamics, one expert $e^*$ happens to receive a slightly higher average routing probability than the others (an essentially inevitable, small asymmetry from random initialization). Because $e^*$ then receives more training examples, its parameters are updated more frequently and hence tend to become relatively *better* at the (shared) task than the other, less-trained experts — this creates a positive feedback loop: a better-performing $e^*$ will tend to be assigned even higher routing probability by the (jointly-trained) router (since sending an input to whichever expert produces the lowest loss for it is exactly what gradient descent on the combined task loss incentivizes the router to learn), which in turn gives $e^*$ even more training signal relative to the other experts, further widening the performance gap. Left unchecked, this positive feedback loop can run away completely, with the router collapsing to route (almost) all inputs to a single dominant expert $e^*$, while the remaining $E - 1$ experts receive vanishingly few training examples and never develop useful specialization — defeating the entire purpose of the MoE architecture (which is to have many specialized experts, each efficiently handling a different subset/aspect of the input distribution).

**Why the router's gradient vanishes in this collapsed regime.** Consider the router's softmax output $p(e \mid x) = \text{softmax}(W_r x)_e$. In the collapsed regime, $p(e^* \mid x) \approx 1$ for the dominant expert and $p(e \mid x) \approx 0$ for all others, for essentially every input $x$ — the router's output distribution is saturated (nearly one-hot), exactly the softmax-saturation phenomenon analyzed in Exercise 12.1. As established there (via the softmax Jacobian, Exercise 9.2), when a softmax distribution becomes extremely peaked/saturated, its local sensitivity to further changes in the pre-softmax logits (here, $W_r x$) collapses toward zero: the Jacobian $\partial p(e \mid x)/\partial(W_r x)$ has entries of order $p(e \mid x)(1-p(e \mid x))$ or similar (directly analogous to $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, Exercise 5.1a, or the general softmax Jacobian $p_i(\mathbf{1}[i = k] - p_k)$, Exercise 9.2), which vanishes as $p \to 0$ or $p \to 1$. So once the router has (even slightly prematurely) become extremely confident that expert $e^*$ is the right choice for essentially all inputs, the *gradient signal that would otherwise push the router to reconsider and explore routing to other experts* is itself attenuated to near-zero by this same saturation — the router receives almost no gradient-based incentive to change its (collapsed) behavior, even though the collapsed behavior is highly suboptimal from the standpoint of overall model capacity utilization. This is a self-reinforcing local-optimum trap: once collapse begins, the vanishing-gradient effect of softmax saturation actively *prevents* the standard task-loss gradient from correcting it.

**The role of the auxiliary load-balancing loss.** To prevent this failure mode, MoE architectures (e.g. Shazeer et al. 2017, Switch Transformer) add an explicit **auxiliary load-balancing loss** — typically of the form $\mathcal{L}_{\text{aux}} = E \cdot \sum_{e=1}^{E} f_e \cdot P_e$, where $f_e$ is the (empirical, non-differentiable) fraction of inputs routed to expert $e$ in a batch and $P_e$ is the (differentiable) average router probability assigned to expert $e$ across the batch — this term is explicitly minimized when routing is balanced ($f_e \approx 1/E$ for all $e$), and its gradient with respect to the router parameters $W_r$ (unlike the collapsed main-task gradient) does *not* vanish under saturation, because $\mathcal{L}_{\text{aux}}$ directly penalizes the imbalance itself (through the differentiable $P_e$ term) rather than depending only on the (saturated, vanishing-gradient) task-performance signal. Adding $\mathcal{L}_{\text{aux}}$ to the total training objective (with a suitable coefficient) actively counteracts the collapse dynamics described above, incentivizing the router to maintain a more balanced/exploratory routing distribution across experts even as individual experts begin to specialize, directly preventing the degenerate single-expert-collapse failure mode.

### Exercise 15.6 — LoRA Parameter Reduction

For a weight matrix $W \in \mathbb{R}^{4096\times4096}$, LoRA introduces $\Delta W = AB$ with $A \in \mathbb{R}^{4096\times r}$, $B \in \mathbb{R}^{r\times4096}$. Compute the parameter reduction ratio for $r = 4, 8, 16, 64$.

**Solution.** **Full fine-tuning parameter count.** Directly fine-tuning the full weight matrix $W \in \mathbb{R}^{4096\times4096}$ requires updating all

$$4096 \times 4096 = 16{,}777{,}216 \approx 16.78\text{M parameters.}$$

**LoRA parameter count.** LoRA freezes the original $W$ and instead learns a low-rank update $\Delta W = AB$ with $A \in \mathbb{R}^{4096\times r}$, $B \in \mathbb{R}^{r\times4096}$:

$$\text{LoRA params} = 4096 \times r + r \times 4096 = 2 \times 4096 \times r = 8192r.$$

**Reduction ratio, for each $r$:**

$$\text{Ratio} = \frac{\text{LoRA params}}{\text{Full params}} = \frac{8192r}{16{,}777{,}216} = \frac{r}{2048}.$$

| $r$ | LoRA params ($8192r$) | Ratio ($r/2048$) | Reduction factor |
|---|---|---|---|
| 4 | 32,768 | $4/2048 \approx 0.00195$ | $\approx 512\times$ fewer params |
| 8 | 65,536 | $8/2048 \approx 0.00391$ | $\approx 256\times$ fewer params |
| 16 | 131,072 | $16/2048 \approx 0.00781$ | $\approx 128\times$ fewer params |
| 64 | 524,288 | $64/2048 \approx 0.03125$ | $\approx 32\times$ fewer params |

At the smallest rank, $r = 4$, LoRA trains only about **0.195**% of the parameters of full fine-tuning for this single weight matrix — a $\sim 512\times$ reduction — while even at the (comparatively large) rank $r = 64$, LoRA still uses only about **3.1**% of the full parameter count, a $32\times$ reduction. This dramatic parameter-efficiency (which, applied across all the weight matrices of a large transformer model, translates into being able to fine-tune models with billions of frozen parameters using only millions of trainable LoRA parameters) is the central practical motivation for LoRA: it makes fine-tuning very large pretrained models feasible on hardware that could not accommodate storing full-fine-tuning gradients/optimizer states for every parameter, while empirically achieving performance competitive with full fine-tuning for many downstream tasks — the implicit assumption being that the necessary *task-specific weight update* $\Delta W$ has intrinsically low rank (a hypothesis supported by empirical observations of the low "intrinsic dimensionality" of fine-tuning updates for large pretrained models, per Aghajanyan et al. 2020 and the original LoRA paper, Hu et al. 2021).

### Exercise 15.7 — Diffusion vs. GAN

Compare diffusion models and GANs on: (a) training stability; (b) mode coverage; (c) generation speed; (d) sample quality. When would you prefer a GAN over a diffusion model?

**Solution.** **(a) Training stability.** GANs are notoriously difficult to train stably: the minimax adversarial game between generator and discriminator (Exercise 15.3) can suffer from oscillating dynamics, non-convergence, and is highly sensitive to the relative capacity/learning-rate balance between $G$ and $D$ — if the discriminator becomes too strong too quickly, the generator's gradient signal (via $\log(1 - D(G(z)))$ or similar) can vanish (another instance of the saturation phenomenon

seen repeatedly throughout this chapter's exercises, e.g. Exercises 12.1, 15.5), stalling training. Diffusion models, by contrast, are trained with a simple, stable *supervised regression* objective (predicting the noise $\epsilon$ added at each forward-process step, or equivalently a denoising score-matching objective) — there is no adversarial game, no min-max saddle-point instability, and training loss decreases smoothly and predictably in a manner much more similar to standard supervised learning. This is widely regarded as diffusion models' single biggest practical advantage over GANs.

**(b) Mode coverage.** GANs are well-documented to suffer from **mode collapse**: because the generator only needs to fool the discriminator (not necessarily cover the full diversity of the true data distribution), it can learn to produce a narrow subset of highly-realistic-looking samples while completely ignoring other legitimate modes of the true data distribution (a phenomenon closely related to the earlier finding, Exercise 15.3, that GAN training targets the Jensen-Shannon divergence — which, unlike some other divergences, does not inherently penalize a generator that fails to cover certain regions of support as long as everything it *does* produce looks realistic). Diffusion models, whose training objective is essentially maximum-likelihood-like (closely related to the ELBO framework of Exercise 15.1, applied to the diffusion process), are empirically far more robust to mode collapse and tend to achieve much better coverage of the full diversity of the training data distribution.

**(c) Generation speed.** This is GANs' major advantage: a trained GAN generates a sample with a **single forward pass** through the generator network ($x = G(z)$ for one noise vector $z$) — extremely fast, suitable for real-time applications. Diffusion models, by contrast, generate samples by iteratively reversing the noising process (Exercise 15.4) over many steps (historically hundreds to a couple thousand denoising steps for the original DDPM formulation, though modern accelerated samplers like DDIM, or distillation-based approaches, have substantially reduced this to as few as tens or even a handful of steps) — even with acceleration, diffusion sampling is typically still meaningfully slower than a single GAN forward pass, making diffusion models less well-suited to strict real-time/low-latency generation requirements.

**(d) Sample quality.** Modern diffusion models (e.g. DALL-E 2/3, Stable Diffusion, Imagen) have generally been shown to match or exceed the best GANs on standard sample-quality metrics (e.g. FID — Fréchet Inception Distance) for high-resolution image generation, particularly excelling in combination with classifier-free guidance and large-scale training, and are currently the dominant paradigm for state-of-the-art text-to-image and other high-fidelity generative modeling tasks; well-tuned GANs (e.g. StyleGAN variants) can still achieve very high sample fidelity/sharpness (sometimes with a distinctive perceptual "crispness" some practitioners still find preferable for certain applications) but typically at the cost of the mode-coverage and training-stability trade-offs noted above.

**When to prefer a GAN over a diffusion model.** GANs remain the preferred choice specifically when **generation speed/latency is critical** — e.g. real-time image synthesis, interactive applications, video-game asset generation, or any deployment setting where the many-step iterative diffusion sampling process is prohibitively slow and the single-forward-pass GAN generation is a hard requirement — provided the application can tolerate (or has specifically engineered around, e.g. via careful architecture and regularization choices such as StyleGAN's design) GANs' characteristic training-stability and mode-coverage challenges. GANs may also be preferred in settings with very limited compute budget for *inference* (since a single forward pass is far cheaper than dozens-to-hundreds of diffusion denoising steps), even if diffusion models are used for *training*-time asset generation that is then distilled into a faster model, or in domains (e.g. some image-to-image translation / style-transfer tasks) where GAN-specific architectural innovations remain particularly

well-suited to the task structure.